# Ingegneria del Software — Soluzione del Tema 13/02/2023

# Esercizio 1

Si consideri la seguente classe `Championship` che memorizza i risultati di un campionato tra diversi `Team`. Si noti che il campionato distingue tra andata e ritorno di ogni partita.

```
public class Championship {
  // Ritorna l'insieme dei team che fanno parte del campionato.
  public /*@ pure @*/ Set<Team> getTeams() {...}

  // Cancella tutti i risultati delle partite fin qui giocate.
  public void clear() {...}

  // Ritorna il team vincente nella partita giocata tra t1 (in casa) e t2 (fuori casa), oppure null
  // se non si e' giocata nessuna partita tra t1 e t2 (t1 e t2 devono essere squadre del campionato).
  // Si noti che il valore restituito potrebbe non coincidere esattamente con t1 o t2, ma deve essere
  // uguale ("equals") a uno dei due.
  public /*@ pure @*/ Team winner(Team t1, Team t2) {...}

  // Inserisce il risultato della partita tra t1 (in casa) e t2 (fuori casa) con vincitore winner
  // (riguardo t1, t2 e winner valgono le stesse note indicate per il precedente metodo).
  // Solleva l'eccezione MatchAlreadyPlayedException se la partita e' gia' stata giocata.
  public void addMatch(Team t1, Team t2, Team winner) throws MatchAlreadyPlayedException {...}

  // Ritorna l'elenco dele squadre con cui la squadra t ha vinto giocando in casa.
  public /*@ pure @*/ List<Team> defeatedAtHome(Team t) {...}

  // Ritorna il punteggio accumulato dalla squadra t (la squadra vincente, in casa o fuori casa,
  // prende un punto).
  public /*@ pure @*/ int numPoints(Team t) {...}
```

### Domanda a)

Specificare in JML il metodo `addMatch`.

### Soluzione

```
//@requires t1!=null && t2!=null && getTeams().contains(t1) &&
//@         getTeams().contains(t2) && !t1.equals(t2) && (winner.equals(t1) || winner.equals(t2))
//@ensures \old(winner(t1,t2)==null) && winner(t1,t2).equals(winner) &&
//@        getTeams().size()==\old(getTeams().size()) &&
//@        getTeams().containsAll(\old(getTeams())) &&
//@        (\forall Team tt1; getTeams().contains(tt1);
//@           (\forall Team tt2; getTeams().contains(tt2);
//@              !tt1.equals(t1) && !tt2.equals(t2) ==> winner(tt1,tt2)==\old(winner(tt1,tt2))))
//@signals(MatchAlreadyPlayedException) \old(winner(t1,t2)!=null) &&
//@        getTeams().size()==\old(getTeams().size()) &&
//@        getTeams().containsAll(\old(getTeams())) &&
//@        (\forall Team tt1; getTeams().contains(tt1);
//@           (\forall Team tt2; getTeams().contains(tt2);
//@              winner(tt1,tt2)==\old(winner(tt1,tt2))))
public void addMatch(Team t1, Team t2, Team winner) throws MatchAlreadyPlayedException {...}
```

### Domanda b)

Specificare in JML i metodi `defeatedAtHome` e `numPoints`.

### Soluzione

```
//@requires t!=null && getTeams().contains(t)
//@ensures (\forall Team t1; ; \result.contains(t1) <==>
//@           (getTeams().contains(t1) && (winner(t,t1)!=null && winner(t,t1).equals(t))) ) &&
//@        (\forall int i; i>=0 && i<\result.size();
//@           (\forall int j; j>i && j<\result.size();
//@              !\result.get(i).equals(\result.get(j))))
public /*@ pure @*/ List<Team> defeatedAtHome(Team t) {...}

//@requires t!=null && getTeams().contains(t)
//@ensures \result == (\numof Team t1; getTeams().contains(t1);
//@                      (winner(t,t1)!=null && winner(t,t1).equals(t)) ||
//@                      (winner(t1,t)!=null && winner(t1,t).equals(t)))
public /*@ pure @*/ int numPoints(Team t) {...}
```

**Domanda c)**

Si consideri una implementazione della classe `Championship` che usa una lista di `Team` per memorizzare le squadre del campionato e un array bidimensionale di `Team` per memorizzare le partite giocate.

```
public class Championship {
  private List<Team> teams;
  private Team[][] res;
  ...
```

La posizione `res[i][j]` vale `null` se la partita tra l'i-esima squadra (in casa) e la j-esima squadra (fuori casa) nella lista `teams` non si è ancora giocata, mentre contiene un riferimento alla squadra vincente se la partita si è giocata.

Per questa implementazione scrivere l'invariante di rappresentazione e la funzione di astrazione.

**Soluzione**

L'invariante di rappresentazione deve dire che i due attributi sono non nulli, che la lista `teams` contiene almeno due squadre, che la dimensione dell'array bidimensionale `res` è coerente con il numero di squadre nella lista `teams`, che non ci sono squadre duplicate nella lista `teams`, che la posizione `[i,j]` (per `i!=j`) della matrice `res` contiene il valore `null` o il riferimento ad una squadra che `equals` l'i-esima o la j-esima squadra in `teams`, ed infine che la diagonale della matrice `res` contiene sempre `null`.

```
//@private invariant
//@ teams!=null && res!=null && teams.size()>=2 &&
//@ res.length==teams.size() &&
//@ (\forall int i; i>=0 && i<teams.size(); res[i].length==teams.size()) &&
//@ (\forall int i; i>=0 && i<teams.size();
//@    (\forall int j; j>=0 && j<teams.size() && i!=j;
//@       !teams.get(i).equals(teams.get(j)) &&
//@       (res[i][j]==null || res[i][j].equals(teams.get(i)) || res[i][j].equals(teams.get(j)))) ) &&
//@ (\forall int i; i>=0 && i<teams.size(); res[i][i]==null)
```

La funzione di astrazione "mappa" gli attributi privati sui metodi base ("ground") usati per la specifica: `getTeams` e `winner`.

```
//@private invariant
//@ getTeams().containsAll(teams) && teams.containsAll(getTeams()) &&
//@ (\forall int i; i>=0 && i<teams.size();
//@    (\forall int j; j>=0 && j<teams.size();
//@       res[i][j]==winner(teams.get(i),teams.get(j)) ) )
```

**Domanda d)**

Si consideri una classe `ReplayableChampionship` che aggiunge alla classe `Championship` il metodo pubblico:
`void replayMatch(Team t1, Team t2, Team winner)`

`ReplayableChampionship` può essere definita come sottoclasse di `Championship` senza violare il principio di sostituzione di Liskov?

**Soluzione**

Si. La classe `ReplayableChampionship` aggiunge un metodo senza ridefinire metodi esistenti, quindi non viola le regole della signature e dei metodi. Inoltre il metodo aggiunto non viola alcuna proprietà invariante perché il suo comportamento è equivalente ad una opportuna sequenza di chiamate dei metodi originali della classe `Championship`, partendo con una chiamata del metodo `clear` e aggiungendo tutte le partite prima aggiunte con il medesimo risultato, tranne la nuova partita che si aggiunge con il nuovo risultato.

# Esercizio 2

Si consideri la seguente classe `Products` che contiene informazioni su un insieme di prodotti. Ogni prodotto è caratterizzato da un identificativo e un nome univoci e un prezzo.

```
public class Products {
  private Map<Integer, String> idToName;
  private Map<String, Integer> nameToPrice;
  private int lastId;

  public Products() {
    idToName = new HashMap<>();
    nameToPrice = new HashMap<>();
    lastId = 0;
  }

  public int addProduct(String name, int price) {
    int assignedId = 0;
    synchronized (idToName) {
      assignedId = lastId++;
      idToName.put(assignedId, name);
    }
    synchronized (nameToPrice) { nameToPrice.put(name, price); }
    return assignedId;
  }

  // Ritorna il prezzo del prodotto avente l'id passato per parametro
  // Se tale prodotto non esiste, ritorna -1
  public int getPriceById(int id) {
    synchronized (idToName) {
      if (idToName.containsKey(id)) {
        String name  = idToName.get(id);
        return nameToPrice.get(name);
      } else {
        return -1;
      }
    }
  }
}
```

**Domanda a)**

Si dica se la classe `Products` è opportunamente sincronizzata per permettere l'invocazione concorrente dei suoi metodi da parte di più thread. In caso negativo, si modifichi la classe per correggere la sua sincronizzazione.

**Soluzione**

La classe *non* è opportunamente sincronizzata, in quanto il metodo `getPriceById` accede alla variabile `nameToPrice` senza acquisire un lock su di essa: di conseguenza, la variabile potrebbe essere modificata durante un'esecuzione concorrente del metodo `addProduct`.

Si noti che proteggere l'accesso a `nameToPrice` nel metodo `getPriceById` con un blocco `synchronized` su di esso *non* sarebbe sufficiente. Occorre rendere atomico l'inserimento di un prodotto nelle due mappe per garantire che il nome del prodotto sia presente in entrambe o in nessuna durante l'invocazione del metodo `getPriceById`.

Entrambe le mappe devono quindi essere aggiornate all'interno del medesimo blocco `synchronized` nel metodo `addProduct`, e il metodo `getPriceById` deve sincronizzarsi sullo stesso oggetto. Nella soluzione proposta, si garantisce atomicità sincronizzando entrambi i metodi su `this`.

```
public class Products {
  private Map<Integer, String> idToName;
  private Map<String, Integer> nameToPrice;
  private int lastId;

  public Products() {
    idToName = new HashMap<>();
    nameToPrice = new HashMap<>();
    lastId = 0;
  }

  public synchronized void addProduct(String name, int price) {
    int assignedId = lastId++;
    idToName.put(assignedId, name);
```

```
    nameToPrice.put(name, price);
    return assignedId;
  }

  // Ritorna il prezzo del prodotto avente l'id passato per parametro
  // Se tale prodotto non esiste, ritorna -1
  public synchronized int getPriceById(int id) throws {
    if (idToName.containsKey(id)) {
      String name  = idToName.get(id);
      return nameToPrice.get(name);
    } else {
      return -1;
    }
  }
}
```

**Domanda b)**

A partire dalla soluzione data alla domanda `(a)`, si modifichi il metodo `getPriceById` in modo che, invece di ritornare -1, sospenda il chiamante se il prodotto non esiste. Se necessario, si indichi come modificare altri metodi.

**Soluzione**

```
public class Products {
  private Map<Integer, String> idToName;
  private Map<String, Integer> nameToPrice;
  private int lastId;

  public Products() {
    idToName = new HashMap<>();
    nameToPrice = new HashMap<>();
    lastId = 0;
  }

  public synchronized void addProduct(String name, int price) {
    int assignedId = lastId++;
    idToName.put(assignedId, name);
    nameToPrice.put(name, price);
    notifyAll(); // Risveglia potenziali thread in attesa di questo id
    return assignedId;
  }

  public synchronized int getPriceById(int id) throws InterruptedException {
    while (!idToName.containsKey(id)) {
      wait(); // Sospende il chiamante se l'id non e' presente
    }
    String name  = idToName.get(id);
    return nameToPrice.get(name);
  }
}
```

**Domanda c)**

Si scriva un metodo `void copyPrices(Products other)` che, per tutti i prodotti di `this` che sono anche contenuti in `other` sostituisce i prezzi contenuti in `this` con i prezzi contenuti in `other`. Si presti particolare attenzione a evitare il rischio di possibili deadlock.

Si ricordano alcuni metodi di `Map` che potrebbero risultare utili per l'implementazione: `Map` ha un costruttore che prende in ingresso un'altra mappa e ne copia il contenuto (chiavi e valori); il metodo `keySet()` ritorna un `Set` con l'insieme delle chiavi; il metodo `put` prende in ingresso una chiave e un valore e aggiunge o sovrascrive il valore associato alla chiave.

**Soluzione**

```
void copyPrices(Products other) {
  Map<String, Integer> copy = null;
  synchronized (other) {
    copy = new HashMap<>(other.nameToPrice);
  }
  synchronized (this) {
    for (String name : copy.keySet()) {
      if (nameToPrice.containsKey(name)) {
        nameToPrice.put(name, copy.get(name));
      }
```

```
        }
    }
}
```

# Esercizio 3

Si considerino le seguenti classi Java

```
public class Shop {
  public List<Product> getProducts();
  ...
}

public class Product {
  public int getPrice();
  public String getName();
  ...
}
```

Si risponda alle seguenti domande facendo uso esclusivamente dei costrutti di programmazione funzionale di Java.

**Domanda a)**

Si completi il metodo `printExpensiveNames`, che stampa il nome di tutti i prodotti dello `Shop s` il cui prezzo è maggiore di 10.

```
public static void printExpensiveNames(Shop s) {
  ...
}
```

**Soluzione**

```
public static void printExpensiveNames(Shop s) {
  s.getProducts().stream()
     .filter(p -> p.getPrice() > 10)
     .map(p -> p.getName())
     .forEach(System.out::println);
}
```

**Domanda b)**

Si completi il metodo `getAllProductsNames`, che ritorna una lista con tutti i prodotti in vendita in almeno uno `Shop` presente nella lista `shops`

```
public static List<String> getAllProductsNames(List<Shop> shops) {
  return ...
}
```

**Soluzione**

```
public static List<String> getAllProductsNames(List<Shop> shops) {
  return shops.stream()
     .flatMap(s -> s.getProducts().stream())
     .map(p -> p.getName())
     .collect(Collectors.toList());
}
```

**Domanda c)**

Si aggiunga alla classe `Shop` un metodo `getBestProducts()`. Il metodo prende in ingresso una funzione `f`, la quale prende in ingresso un `Product` e ritorna un intero rappresentante l'indice di gradimento (rating) di quel prodotto. Il metodo stampa il nome di tutti i prodotti con rating maggiore di 10.

Si ricorda che in Java una funzione è rappresentata da un'interfaccia funzionale `Function<T, R>`, con un unico metodo: `R apply(T input)`.

**Soluzione**

```
public void getBestProducts(Function<Product, Integer> f) {
  this.getProducts().stream()
     .filter(p -> f.apply(p) > 10)
     .map(p -> p.getName())
     .forEach(System.out::println);
}
```

# Esercizio 4

Si considerino le seguenti dichiarazioni di classi Java.

```
abstract class Box {
  protected String secret;
  protected String content() { return secret; }
  public void show(Box p) {
    System.out.print("Box.show(Box)");
    System.out.println(p.content());
  }
  public Box(String param) {
    secret = param + " box ";
  }
}

class SteelBox extends Box {
  public void show(SteelBox p) {
    System.out.println(p.content());
  }
  public void show(Box p) {
    System.out.println(content() + "&& " + p.content());
  }
  public SteelBox(String param) {
    super(param + " steel ");
  }
}

class LockedBox extends SteelBox {
  protected String content() { return "secret! "; }
  public void show(LockedBox p) {
    System.out.println(content() + "and " + p.content());
  }
  public void show(Box p) {
    System.out.println(content() + "with " + p.content());
  }
  public LockedBox(String param) {
    super(param + " locked ");
  }
}
```

e il seguente frammento di codice, in cui le righe sono numerate per comodità:

```
1  Box b,s,l;
2  SteelBox sb;
3  LockedBox lb;
4  b = new Box(" b");
5  s = new SteelBox(" s");
6  l=s;
7  s.show(l);
8  lb=s;
9  l = new LockedBox(" l");
10 l.show(s);
11 sb = new SteelBox(" sb");
12 lb = new LockedBox(" lb");
13 sb.show(s);
14 sb.show(l);
15 sb.show(sb);
16 sb.show(lb);
17 lb.show(s);
18 lb.show(l);
19 lb.show(sb);
20 lb.show(lb);
```

### Domanda a)

Si indichino i numeri di ogni riga del frammento sopra riportato che genera un errore, precisando se l'errore si manifesti in fase di compilazione o in fase di esecuzione. Motivare brevemente la risposta.

### Soluzione

Riga 4: Compilazione, una classe astratta non può essere istanziata.
Riga 8: Compilazione, assegnamento polimorfo errato.

**Domanda b)**

Supponendo che tutte le righe che provocano errori in fase di compilazione o di esecuzione siano state rimosse, mostrare per ogni istruzione di stampa quale stringa viene prodotta in output, motivando brevemente la propria risposta. Riportare **sempre** il numero di riga di fianco alla risposta.

**Soluzione**

```
 s steel  box &&  s steel  box


secret! with  s steel  box


sb steel  box &&  s steel  box
sb steel  box && secret!
sb steel  box
secret!
secret! with  s steel  box
secret! with secret!
sb steel  box
secret! and secret!
```